第一章 绪论

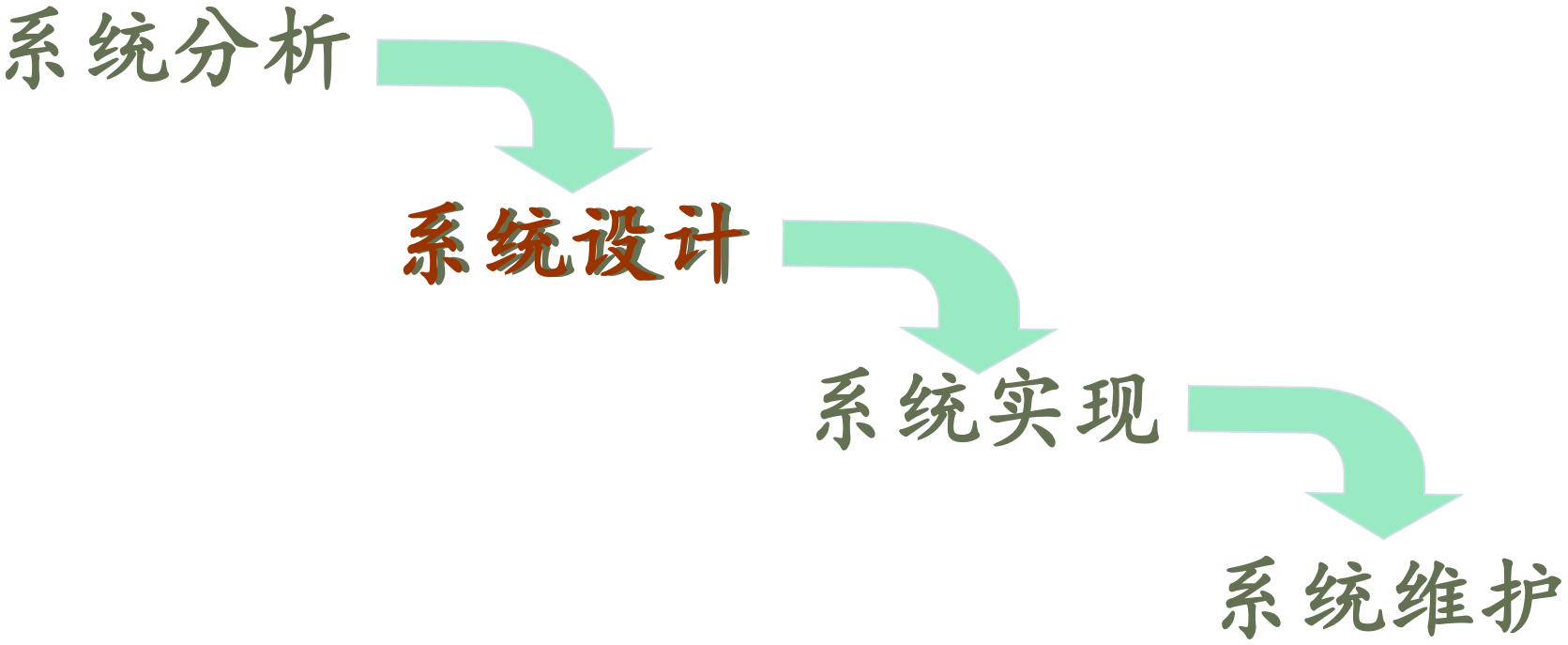
1.1 数据结构讨论的范畴

1.2 基本概念

1.3 算法和算法的量度

1.1 数据结构讨论的范畴

(数据结构在软件开发中的地位)



Niklaus Wirth

Algorithm + Data Structures = Programs

程序设计： 为计算机处理问题编制一组指令集

数据结构： 问题的数学模型

算法： 处理问题的策略

例如： 数值计算的程序设计问题

结构静力分析计算

—— 线性代数方程组

全球天气预报

—— 环流模式方程
(球面坐标系)

数学模型可以用数学方程加以描述

非数值计算的程序设计问题

例一：求一组(n个)整数中的最大值

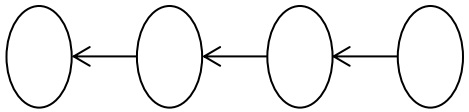
算法：？ 基本操作是“**比较两个数的大小**”

模型：？ 取决于**整数值的范围**

例二：旅馆客房的管理

算法：？　先进先出

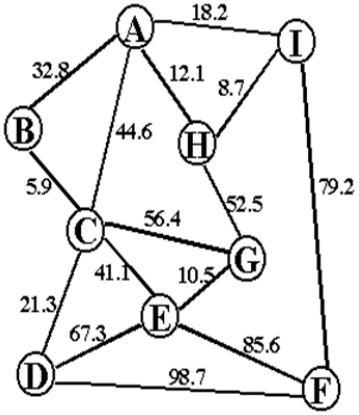
模型：？　队列

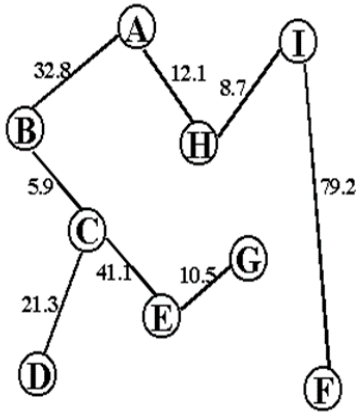


例三：铺设城市的煤气管道

算法：？　如何规划使得总投资花费最少？

模型：？　图





概括地说，

数据结构是一门讨论“描述现实世界实体的**数学模型**(非数值计算)及其上的操作在计算机中**如何表示和实现**”的学科。

1.2 基本概念

一、数据与数据结构

二、数据类型

三、抽象数据类型

一、数据与数据结构

数据：

所有能**被输入**到计算机中，且能被计算机**处理的符号(数值、字符等)**的集合。

是**计算机操作的对象**的总称。

数据元素：

是数据（集合）中的一个“个体”，在计算机中通常作为一个整体进行考虑和处理。是数据结构中讨论的基本单位。

如：整数“5”，字符“N”等。

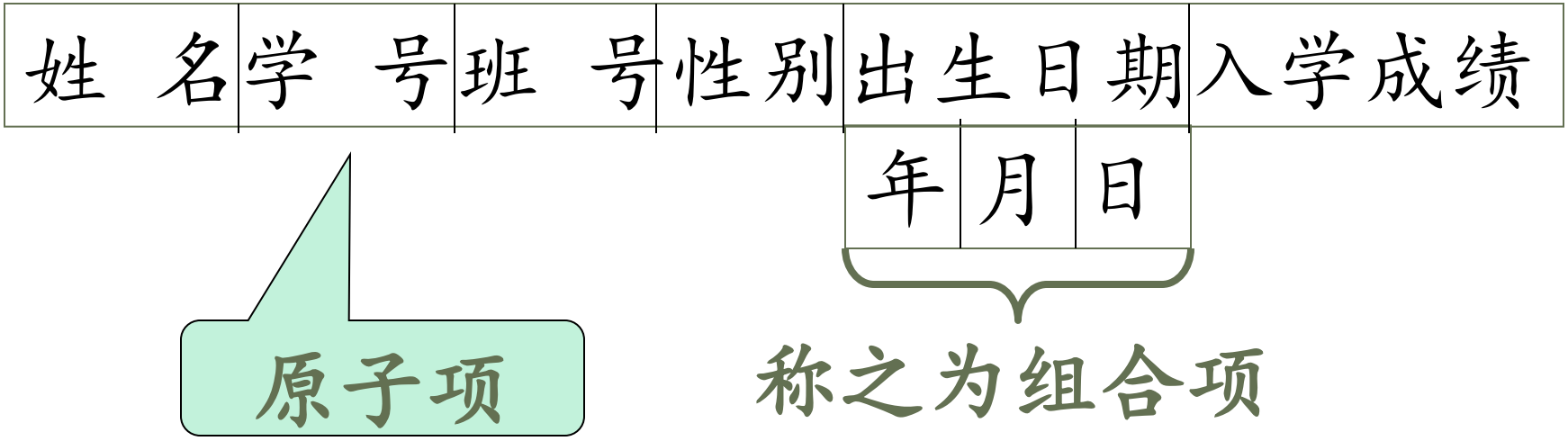
----是不可分割的“原子”

数据元素也可以由若干款项构成。

例如：描述一个学生的数据元素

其中每个款项称为一个“数据项”

它是数据结构中讨论的最小单位



数据结构：

有一个特性相同的数据元素的集合，如果在数据元素之间存在一种或多种特定的关系，则称为一个数据结构。

带结构的数据元素的集合

指的是数据元素之间存在的关系

例如，当用**三个 4 位的十进制数**表示一个含 **12 位数的十进制数时，**

例如：

3214,6587,9345 — **a1**(3214),**a2**(6587),**a3**(9345)

则在数据元素 a1、a2 和 a3 之间存在着

“次序”关系 **{ <a1, a2>、<a2, a3> }**

序偶

3214，6587，9345　　6587，3214，9345

a1　a2　a3　　a2　a1　a3

又例，在 2 行 3 列的二维数组中六个元素 {a1, a2, a3, a4, a5, a6} 之间存在两个关系：

a1	a2	a3
a4	a5	a6

行的次序关系：

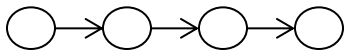
row = {<a1,a2>,<a2,a3>,<a4,a5>,<a5,a6>}

列的次序关系：

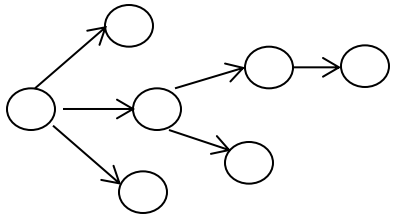
col = {<a1,a4>,<a2,a5>,<a3,a6>}

从**关系**或**结构**分，**数据结构**可归结为以下**四类**：

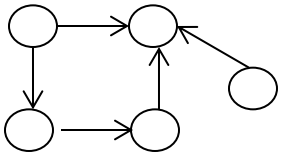
线性结构



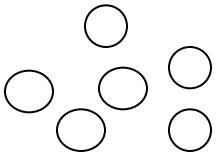
树形结构



图状结构



集合结构



数据结构的形式定义描述为：

数据结构是一个二元组

Data_Structures = (D, S)

其中：D 是**数据元素的有限集**，

S 是 D上**关系的有限集**。

例如:定义“班集体”为一个数据结构

$$Class = (D, S)$$

$$D = \{ a, b_1,\ldots,b_n, c_1,\ldots c_n, d_1,\ldots d_n \}$$

$$S = \{ R_1, R_2 \}$$

$$R1 = \{ <a, b_1>,<a, c_1>,<a, d_1>\}$$

$$R2 = \{ <b_1, b_j>, <c_1, c_j>, <d_1, d_j> \mid j = 2, 3, \ldots, n \}$$

数据结构包括“**逻辑结构**”和“**物理结构**”两个方面(层次):

逻辑结构 是对数据元素之间的逻辑关系的描述，它可以用一个数据元素的集合和定义在此集合上的若干关系来表示;

物理结构 是逻辑结构在计算机中的表示和实现，故又称“**存储结构**”。

数据的存储结构

—— 逻辑结构在存储器中的映象

“数据元素”的映象 ？

“关系”的映象 ？

数据元素的映象方法：

用二进制位(bit)的位串表示数据元素

$$(321)_{10} = (501)_8 = (101000001)_2$$

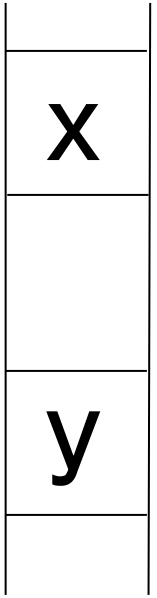
$$A = (101)_8 = (001000001)_2$$

关系的映象方法:(表示<x, y>的方法)

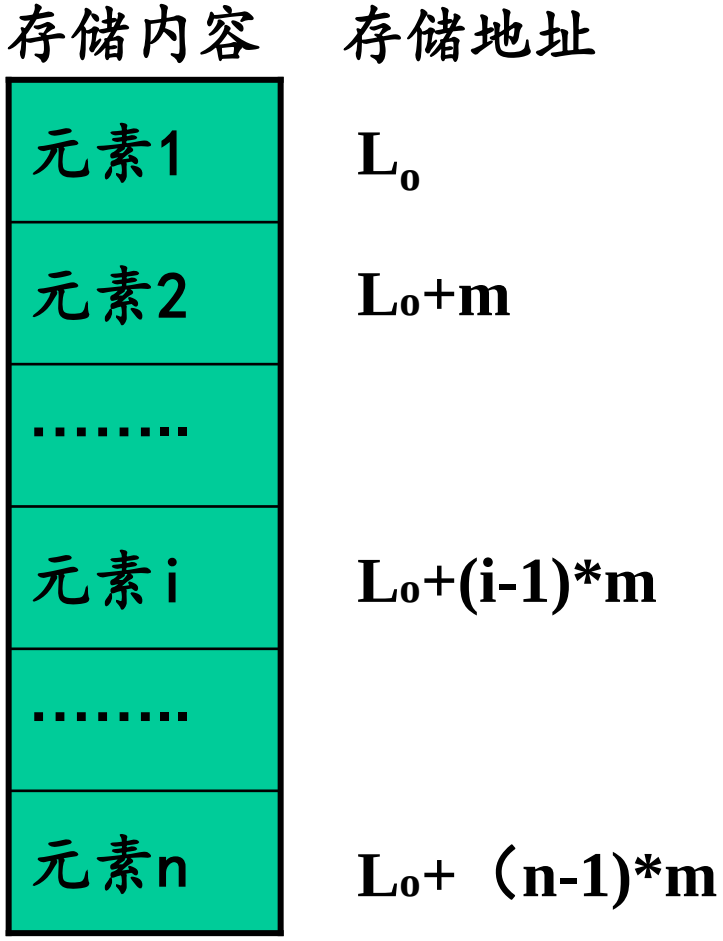
顺序映象: 以相对的存储位置表示后继关系

例如:令 y 的存储位置和 x 的存储位置之间差一个常量 m

而 m 是一个隐含值，**整个存储结构中只含数据元素本身的信息**



顺序存储

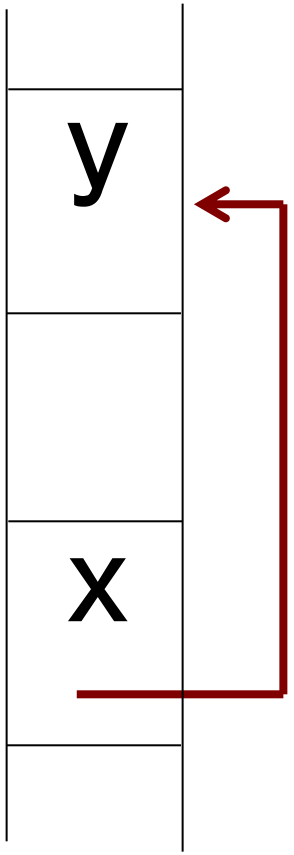


Location(元素i) = Lo +（i-1)*m

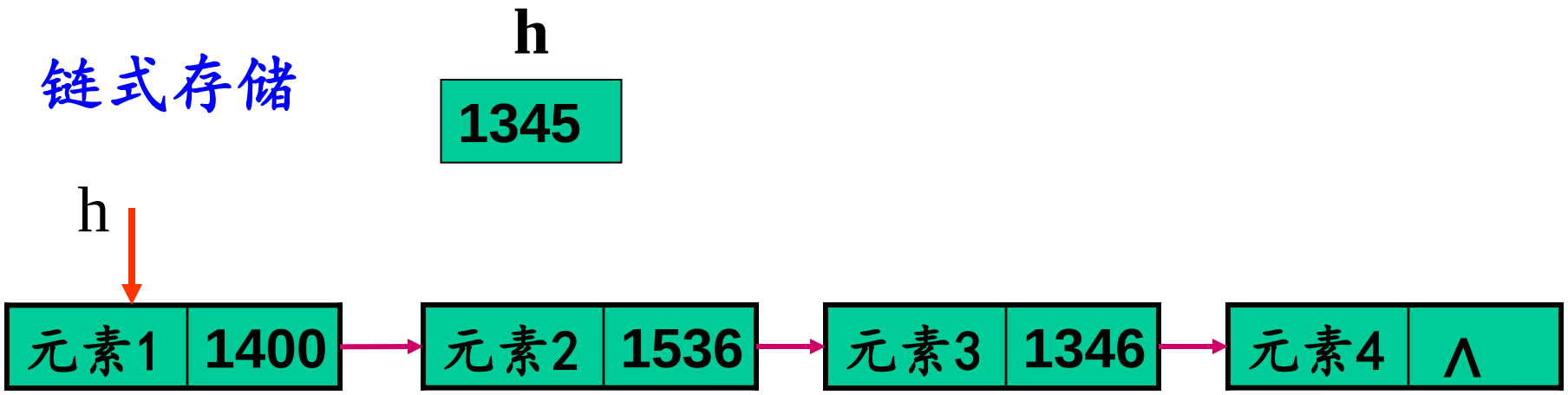
链式映象

以附加信息(指针)表示后继关系

需要用一个和 x 在一起的**附加信息**指示 y 的存储位置



链式存储



存储地址	存储内容	指针
1345	元素1	1400
1346	元素4	∧
…….	…….	…….
1400	元素2	1536
…….	…….	…….
1536	元素3	1346

在不同的编程环境中，

存储结构可有不同的描述方法，

当用高级程序设计语言进行编程时，通常可用高级编程语言中提供的数据类型描述之。

例如：

以三个带有次序关系的整数表示一个长整数时，可利用 C 语言中提供的整数数组类型，

定义长整数为：

```
typedef   int   Long_int[3];
```

数据结构的三个方面：

- 数据的逻辑结构
  - 线性结构
    - 线性表
    - 栈
    - 队
  - 非线性结构
    - 树形结构
    - 图形结构
- 数据的存储结构
  - 顺序存储
  - 链式存储
- 数据的运算：检索、排序、插入、删除、修改等

二、数据类型

在用高级程序语言编写的程序中，必须对程序中出现的每个变量、常量或表达式，**明确说明**它们所属的**数据类型**。

例如，C 语言中提供的**基本数据类型**有：

整型　int

浮点型　float

双精度型　double

} **实型**

字符型　char

逻辑型　bool （C++语言）

不同类型的变量，其所能取的**值的范围**不同，所能**进行的操作**不同。

例如：整型

值的范围是：-32768 ~ 32767

操作是：+，-，*，/，%

数据类型 是一个 **值的集合** 和定义在此集合上的 **一组操作** 的总称。

汇编语言中十进制表示的数据98.65、9.6E3等，它们是二进制数据的抽象；

高级语言中，给出更高一级的数据抽象，如整型、实型、字符型等；

还可以进一步定义更高级的数据抽象，如各种表、队、栈、树、图、窗口、管理器等复杂的**抽象数据类型**。

三、抽象数据类型

（Abstract Data Type 简称ADT）

是指一个数学模型以及定义在此数学模型上的一组操作。

例如：

“整数”是一个抽象数据类型。

其数学特性和具体的计算机或语言无关。抽象”的意义在于强调数据类型的数学特性。

抽象数据类型还包括用户在设计软件系统时自己定义的数据类型。

例如，定义抽象数据类型“复数”

ADT Complex {

数据对象：

D＝{e1,e2 ｜ e1,e2∈RealSet }

数据关系：

R1＝{<e1,e2> | e1是复数的实数部分,

| e2 是复数的虚数部分 }

基本操作：

InitComplex(&Z, v1, v2)

操作结果：构造复数 Z, 其实部和虚部分别被赋以参数 v1 和 v2 的值。

DestroyComplex(&Z)

操作结果：复数Z被销毁。

GetReal(Z, &realPart)

初始条件：复数已存在。

操作结果：用realPart返回复数Z的实部值。

GetImag(Z, &ImagPart)

初始条件：复数已存在。

操作结果：用ImagPart返回复数Z的虚部值。

Add(z1,z2, &sum)

初始条件：z1, z2是复数。

操作结果：用sum返回复数z1, z2 的和值。

……

} ADT Complex

$$z = \frac{(8+6i)(4+3i)}{(8+6i)+(4+3i)}$$

```
# include <iostream>
# include "complex.h"
void main()
{
    … …
}
```

$$z = \frac{(8+6i)(4+3i)}{(8+6i)+(4+3i)}$$

```
complex z1,z2,z3,z4,z;
float RealPart,ImagPart;
InitComplex(z1,8.0,6.0);
InitComplex(z2,4.0,3.0);
Add(z1,z2,z3);
Multiply(z1,z2,z4);
if (Division (z4,z3,z))
{       GetReal (z, RealPart);
        GetImag (z, ImagPart);
 }//if
```

抽象数据类型的描述方法

抽象数据类型可用（**D**，**S**，**P**）三元组表示

其中，D 是数据对象，

S 是 D 上的关系集，

P 是对 D 的基本操作集。

ADT 抽象数据类型名 {

数据对象：〈数据对象的定义〉

数据关系：〈数据关系的定义〉

基本操作：〈基本操作的定义〉

} ADT 抽象数据类型名

其中基本操作的定义格式为：

基本操作名（参数表）

初始条件：〈初始条件描述〉

操作结果：〈操作结果描述〉

赋值参数 只为操作提供输入值；

引用参数 以&打头，除可提供输入值外，还将返回操作结果。

初始条件 描述了操作执行之前数据结构和参数应满足的条件，若不满足，则操作失败，并返回相应出错信息。

操作结果 说明了操作正常完成之后，数据结构的变化状况和应返回的结果。若初始条件为空，则省略之。

抽象数据类型的表示和实现

用标准C语言实现ADT描述时，有两个方面：

１）通过结构体将int、float等固有类型组合到一起，构成一个结构类型，再用typedef为该类型或该类型指针重新起一个名字。

２）用C语言函数实现各操作。基本操作包括插入、删除、更新、查找、排序等 。

例如，对以上定义的复数

```
// -----存储结构的定义
typedef  struct
{   float realpart;
    float imagpart;
}complex;

// -----基本操作的函数原型说明

void InitComplex ( complex &Z,
                   float realval, float imagval );
//构造复数 Z, 其实部和虚部分别被赋以参数
// realval 和 imagval 的值
```

```
void GetReal( complex Z , &RealPart);
  // 返回复数 Z 的实部值RealPart

void GetImag( complex Z , &ImagPart);
  // 返回复数 Z 的虚部值ImagPart

void Add( complex z1, complex z2,
                                complex  &sum );
  // 以 sum 返回两个复数 z1, z2 的和
```

```
// -----基本操作的实现

void Add( complex z1, complex z2,
                                complex  &sum )
{ // 以 sum 返回两个复数 z1, z2 的和
   sum.realpart = z1.realpart + z2.realpart;
   sum.imagpart = z1.imagpart + z2.imagpart;
}

{ 其它省略 }
```

1.3 算法和算法的衡量

一、算法

二、算法设计的原则

三、算法效率的衡量方法和准则

四、算法的存储空间需求

一、算法

算法是为了解决某类问题而规定的一个有限长的**操作序列**。一个算法必须满足以下**五个重要特性**：

1. **有穷性**　2. **确定性**　3. **可行性**

4. **有输入**　5. **有输出**

1．**有穷性**　对于任意一组合法输入值，在执行**有穷步骤**之后一定能结束，即：算法中的每个步骤都能在**有限时间**内完成；

2．**确定性**　对于**每种情况**下所应执行的操作，在算法中都有**确切**的规定，使算法的执行者或阅读者都能明确其含义及如何执行。**并且在任何条件下，算法都只有一条执行路径；**

3. 可行性 算法中的所有操作都必须足够基本，都可以通过已经实现的基本操作运算有限次实现之；

4. 有输入 作为算法加工对象的量值，通常体现为算法中的一组变量。有些输入量需要在算法执行过程中输入，而有的算法表面上可以没有输入，实际上已被嵌入算法之中；

5. 有输出 它是一组与“输入”与确定关系的量值，是算法进行信息加工后得到的结果，这种确定关系即为算法的功能。

二、算法设计的原则

设计算法时，通常应考虑达到以下目标：

1. 正确性
2. 可读性
3. 健壮性
4. 高效率与低存储量需求

1. 正确性

首先，算法应当满足以特定的“规格说明”方式给出的需求。

其次，对算法是否“正确”的理解可以有以下四个层次：

a. 程序中不含语法错误；

b. 程序对于几组输入数据能够得出满足要求的结果；

c．程序对于精心选择的、典型、苛刻且带有刁难性的几组输入数据能够得出满足要求的结果；

d．程序对于一切合法的输入数据都能得出满足要求的结果；

通常以第 c 层意义的正确性作为衡量一个算法是否合格的标准。

2. 可读性

算法主要是为了人的**阅读与交流**，其次才是为计算机执行。因此算法应该**易于人的理解**；另一方面，晦涩难读的程序易于隐藏较多错误而难以调试；

3. 健壮性

当输入的数据**非法**时，算法应当恰当地作出反映或**进行相应处理**，而不是产生莫名奇妙的输出结果。并且，**处理出错的方法**不应是中断程序的执行，而应是**返回一个表示错误或错误性质的值**，以便在更高的抽象层次上进行处理。

4. 高效率与低存储量需求

通常，效率指的是算法执行时间；存储量指的是算法执行过程中所需的最大存储空间。两者都与问题的规模有关。

描述算法的语言选择

类C语言：

类语言是接近于高级语言而又不是严格的高级语言，具有高级语言的一般语句设施，撇掉语言中的细节，以便把注意力主要集中在算法处理步骤本身的描述上。

对C语言作以下描述：

1. 预定义常量和类型

```
# define  TRUE  1
# define  FALSE  0
# define  OK  1
# define  ERROR  0
typedef  int  Status;
typedef  int  ElemType;
```

对C语言作以下描述：

2. 基本操作的实现使用函数来描述：

```
[函数类型] 函数名（[形式参数及说明]）
{
        内部数据说明;
        执行语句组;
} /*函数名*/
```

形式参数中以&打头的参数为引用参数。

对C语言作以下描述：

3. 赋值语句

（1）简单赋值：

变量名=〈表达式〉、 变量名++ 、变量名--

（2）串联赋值

变量名1 = 变量名2 = … = 变量名k =〈表达式〉

（3）成组赋值

（变量1，…，变量名k）=〈表达式1，…，表达式k〉

（4）交换赋值

变量1 ↔ 变量2

对C语言作以下描述：

4. 条件选择语句

1) if (<表达式>) 语句;

2) if (<表达式>)

语句1;

else

语句2;

对C语言作以下描述：

情况语句

```
switch (<表达式>)
 {
        case 判断值1:     语句组 1;      break;
        case 判断值2:     语句组 2;      break;
        ……
        case  判断值n:    语句组n;       break;
       [default:          语句组;        break;]
 }
```

对C语言作以下描述：

5. 循环语句

for (<表达式1>；<表达式2>；<表达式3>)
{循环体语句；}

while (<条件表达式>)
{循环体语句；}

do { 循环体语句 }
while (<条件表达式>)

对C语言作以下描述：

6. 输入、输出函数： printf, scanf, cin, cout
7. 其它一些语句： return, break, exit
8. 注释语句： //
9. 一些基本的函数

   求最大值max，求最小值min，求绝对值abs

   求不足整数值floor，求进位整数值ceil

三、算法效率的衡量方法

通常有**两种**衡量算法效率的方法：

① 事后统计法

缺点：1、必须执行程序

2、其它因素掩盖算法本质

② 事前分析估算法

和算法执行时间相关的因素：

1. 算法选用的策略
2. 问题的规模
3. 编写程序的语言
4. 编译程序产生的机器代码的质量
5. 计算机执行指令的速度

一个特定**算法的“运行工作量”**的大小，只依赖于问题的规模（通常用整数量n表示），或者说，**它是问题规模的函数。**

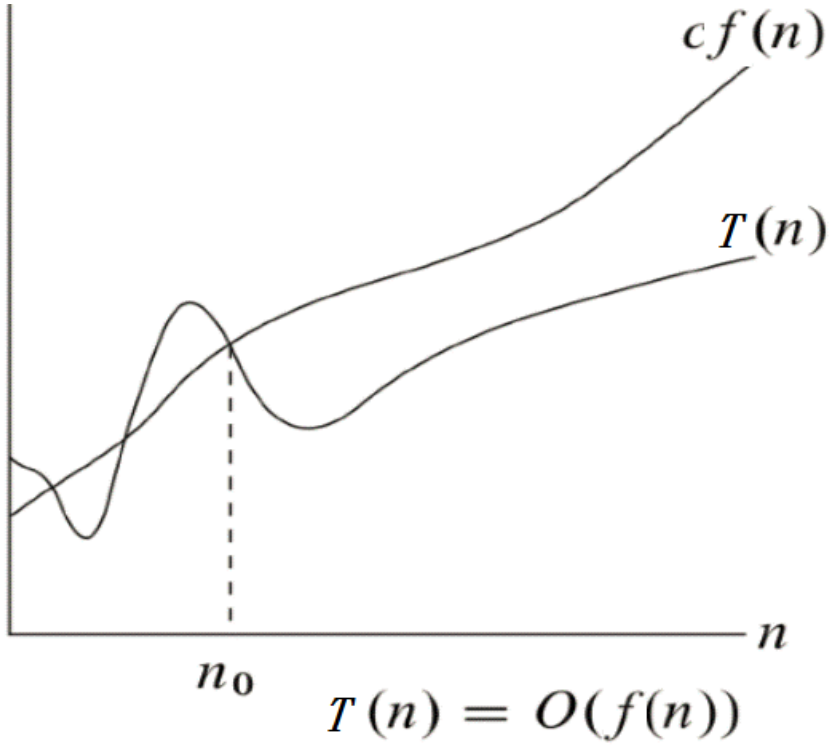
假如，随着问题规模 n 的增长，**算法执行时间的增长率和 f(n) 的增长率相同**，则可记作：



$$T(n) = O(f(n))$$

称T (n) 为算法的(渐近)时间复杂度

T(n)=O(f(n))的曲线图



O的形式定义：若f(n)是正整数n的一个函数，则T(n)=O(f(n))表示存在一个正的常数c，使得当$n \geq n_0$时都满足$|T(n)| \leq c|f(n)|$。

如何估算

算法的时间复杂度?

算法 = 控制结构 + 原操作

（固有数据类型的操作）

算法的执行时间 =

∑原操作(i)的执行次数×原操作(i)的执行时间

算法的执行时间

与

原操作执行次数之和 成正比

从算法中选取一种对于所研究的问题来说是 **基本操作** 的原操作，以该基本操作 **在算法中重复执行的次数** 作为算法运行时间的衡量准则。

例如：给出基本操作 x=x+1

（1）x=x+1；时间复杂度为O(1)，称为常数型；

（2）for（i=1；i<= n；i++）x=x+1；

时间复杂度为O(n)，称为线性型；

数据结构中常用的时间复杂度有7个：

O(1) 常数型　　O(n) 线性型　　$O(n^2)$ 平方型

$O(n^3)$ 立方型　　$O(2^n)$ 指数型　　$O(\log_2 n)$ 对数型

$O(n\log_2 n)$ 二维型

例一

两个矩阵相乘

```
void mult(int a[], int b[], int& c[] )
{ // 以二维数组存储矩阵元素，c 为 a 和 b 的乘积
   for (i=1; i<=n; ++i)
      for (j=1; j<=n; ++j)
      {  c[i,j] = 0;
         for (k=1; k<=n; ++k)
            c[i,j] += a[i,k]*b[k,j];
      } //for
} //mult
```

基本操作：**乘法**操作

时间复杂度：$O(n^3)$

例二 选择排序

```
void select_sort(int& a[], int n)
{ // 将 a 中整数序列重新排列成自小至大有序的整数序列。
  for ( i = 0;  i< n-1;  ++i )
  {   j = i;   // 选择第 i 个最小元素
      for ( k = i+1;  k < n;  ++k )
          if (a[k] < a[j] )  j = k;
      if ( j != i )  a[j] ←→ a[i]
  }
}
```

基本操作：**比较**（数据元素）**操作**

时间复杂度：$\mathbf{O(n^2)}$

例三 起泡排序

```
void bubble_sort(int& a[], int n)
{ // 将 a 中整数序列重新排列成自小至大有序的整数序列。
  for(i=n-1, change=TRUE; i>1 && change; --i)
  {  change = FALSE;  //change为元素进行交换标志
     for (j=0;  j<i;  ++j)
        if (a[j] > a[j+1])
           { a[j] ←→ a[j+1];   change = TRUE ;}
  } // 一趟起泡
} // bubble_sort
```

基本操作：**赋值操作**

时间复杂度：$O(n^2)$

四、算法的存储空间需求

算法的空间复杂度定义为：

$$S(n) = O(g(n))$$

表示随着问题规模 n 的增大，算法运行所需存储量的增长率与 g(n) 的增长率相同。

算法的存储量包括:

1. 输入数据所占空间

2. 程序本身所占空间;

3. 辅助变量所占空间。

若**输入数据**所占空间只取决于问题本身，**和算法无关**，则只需要分析除输入和程序之外的**辅助变量**所占**额外空间**。

若所需额外空间相对于输入数据量来说是常数，则称此算法为**原地工作**。

若所需存储量依赖于特定的输入，则通常按最坏情况考虑。

本章学习要点

1. 熟悉各名词、术语的含义，掌握基本概念。

2. 理解算法五个要素的确切含义。

3. 掌握估算算法时间复杂度的方法。